SquadStation
Updated Frontend Build Guide

Screen-by-screen client specification aligned with the resolved backend audit findings

August 2026 - Revision 2

Scope note

This revision updates frontend behaviour and API expectations for the resolved issues previously identified in the backend review. It intentionally does not inspect or describe any remaining backend work. Use the deployed OpenAPI contract as the final source of exact field names and response envelopes.

1. What changed in this revision

The guide now treats authentication, chat access, list loading, and concurrent actions as production-grade flows rather than optimistic happy paths.

Area	Frontend requirement
OTP	Only treat verification as successful when the server returns tokens. Show invalid or expired code errors without navigating.
Tokens	Replace both tokens after refresh. Refresh tokens may rotate and must never be reused after replacement.
Lists	Use cursor or page based loading, bounded page sizes, and an explicit Load more control.
WebSocket	Connect only on a chat screen, subscribe only after access is confirmed, and handle reconnect/error states.
Validation	Validate client-side for immediate feedback, but always render server validation and conflict errors.
Marketplace	Use the gateway route /api/marketplace/**, not /marketplace-service/**.

2. Global setup

2.1 Environment configuration

Setting	Development value	Rule
REST_API_BASE_URL	http://localhost:8080	All REST calls use the API Gateway.
WS_BASE_URL	http://localhost:8085	Use only if the gateway has not been configured to proxy WebSocket traffic.

Read these values from build-time environment configuration. Do not hard-code production URLs, secrets, refresh tokens, or service-to-service routes into the frontend.

2.2 Authentication and refresh

- Keep access and refresh tokens in the chosen secure client storage strategy. Never put either token in URLs, logs, analytics events, or error reports.

- Attach **Authorization: Bearer <accessToken>** to authenticated REST requests and to the STOMP CONNECT frame.

- On one 401 response, make exactly one refresh attempt. Persist the complete returned token pair before retrying the original request once.

- If refresh fails, clear auth state, disconnect any STOMP client, and navigate to Login. Do not retry refresh in a loop.

3. Shared UI behaviour

3.1 Error model

Show safe, human-readable server messages for expected 400, 401, 403, 404, and 409 responses. For 5xx or network failures, show a generic retryable message. Never display stack traces, raw tokens, or internal service names.

Response	Client behaviour
400 validation	Keep entered values, show field-level errors where possible, and focus the first invalid field.
401	Run the single refresh flow. If it fails, sign out.
403	Show access denied; leave the current screen or return to a safe route.
404	Show not found or removed; remove stale item from local state.
409	Treat as a race/idempotency result. Re-fetch the resource and show the current state.
429	Respect Retry-After when supplied; disable resend/submit until the cooldown ends.

3.2 Client-side validation

Feature	Minimum UI checks
Signup and login	Required college email; trim whitespace; normalise for display only according to backend policy.
OTP	Exactly six numeric digits; prevent repeated submit while pending; show resend cooldown.
Trips	Required source, boarding station, mode, and future travel date/time. Keep destination and vehicle number optional.
Listings	Required type, source, destination, future travel date, positive price and positive whole-number quantity. Cap description input length to the API contract.
Chat	Required non-blank content; enforce the server message limit in the composer; disable Send while a message is pending.

Client validation improves UX only. The server remains authoritative, so every form must still handle validation responses.

3.3 Pagination and local state

- Request the first bounded page for listings and histories; request later pages only after an explicit user action or controlled infinite-scroll trigger.
- Store item IDs and cursor/page metadata. Deduplicate items when a WebSocket event arrives for an item already in the loaded page.
- Reset the page/cursor when filters change. Do not fetch an entire conversation or marketplace catalogue at screen mount.

4. Authentication screens

Screen 1: Splash / Auto-login

Call **GET /api/users/me** only when an access token exists. A valid response goes to Home. A 401 triggers the one-time refresh flow; failed refresh goes to Login.

Screen 2: Signup

Request	Success	Important states
POST /api/users/signup name, collegeEmail, branch, year	Navigate to OTP Verification with email and flow=signup.	Disable submit while pending. Render 400 validation, 409 existing-account, 429 cooldown, and generic network errors.

Screen 3: OTP Verification

Request	Success	Failure
POST /api/users/verify-otp collegeEmail, otp	Only when a token pair is returned: save both tokens and navigate to Home.	For invalid/expired/locked code, remain on this screen, clear the code, announce the error, and allow retry only when not cooling down.

Resend must call the original signup or login operation once, then start a visible countdown. Avoid automatic resend and do not reveal whether a private account exists beyond the backend's approved error wording.

Screen 4: Login

POST **/api/users/login** with collegeEmail. On success navigate to OTP Verification with flow=login. Handle no-account, validation, throttling, and network failure inline.

Logout

POST **/api/users/logout** with the current refresh token, then clear local auth state and disconnect WebSocket sessions regardless of the response. A logout request must not block local sign-out.

5. Trip and group screens

Screen 5: Home

Show navigation for Post a Trip, Browse Marketplace, My Listings, Profile, and available group/chat entry points. Avoid showing a My Trips screen unless the backend exposes a user-owned trips endpoint.

Screen 6: Post a Trip

Request	Success	Client handling
POST /api/trips sourcePoint, boardingStation, destination?, mode, vehicleNumber?, travelDateTime	Use {myTrip, immediateMatches} to navigate to Trip Matches.	Do not send userId; identity comes from the access token. Validate future date/time and prevent duplicate submits.

Screen 7: Trip Matches

- Re-fetch via GET /api/trips/{tripId}/matches when revisiting the screen. Render empty state when no matches exist.
- Create a group with POST /api/groups using the selected trip context. If a 409 conflict occurs, re-fetch matches/group state and offer Join instead.
- Join using POST /api/groups/{groupId}/join. On a trip-mismatch/access error, keep the user on the screen and explain that an eligible matching trip is required.
- If group availability is unavailable, do not guess that no group exists; show Retry.

Screen 8: Group Chat

Initial data	Actions	Exit
GET /api/chat/group/{groupId}/history using pagination; GET /api/groups/{groupId}/members.	Connect STOMP, subscribe after access confirmation, render new messages, and send through /app/chat.sendMessage.	POST /api/groups/{groupId}/leave; disconnect socket and go Home.

6. Marketplace screens

Screen 9: Browse Marketplace

All marketplace traffic goes through **/api/marketplace/**** at the gateway. Load a bounded first page of active listings. Search with source, destination, and travelDate; reset pagination when any filter changes.

Screen 10: Post a Listing

POST **/api/marketplace/listings** with listingType, status, ticketClass, source, destination, travelDate, price, quantity, and description. Keep API-assigned fields such as id, active, postedAt, and postedByUserId out of the request.

Screen 11: Listing Detail

Viewer	Available actions	Required behaviour
Non-owner	Express Interest; start listing conversation if allowed.	Disable interest while pending. On 409, re-fetch and render already-interested state. On inactive/sold response, remove actions and show unavailable.
Owner	View interested users; mark as sold; delete listing.	Optimistically update only after safe rollback support, otherwise re-fetch on success. Confirm destructive deletion in the UI.

Use the deployed contract for the exact mark-sold and delete methods/paths. Do not retain the previous guide's inconsistent PATCH-delete versus PATCH-sold assumptions.

Screen 12: Listing Chat

- Start or resume with POST /api/chat/listing-conversations/start?listingId={id}. Treat concurrent starts as idempotent: a conflict should re-fetch or use the returned existing conversation.
- Load GET /api/chat/listing-conversations/{conversationId}/history in pages and subscribe to the authorised listing conversation destination after access succeeds.
- If the listing becomes unavailable or access is revoked, stop message sending, unsubscribe, and return to Listing Detail with an explanatory state.

7. WebSocket integration

The client may initiate the connection only on an active chat screen. Server-side subscription authorization is authoritative: a successful CONNECT never guarantees access to every topic.

7.1 Connection lifecycle

Moment	Frontend action
Mount chat screen	Create SockJS/STOMP client, send Authorization on CONNECT, render Connecting state.
Connected	Subscribe to the specific authorised conversation/group and /user/queue/errors. Then load or reconcile the newest history page.
Network loss	Render Disconnected state. Retry with capped exponential backoff only while the chat screen remains active.
403/access error	Unsubscribe, stop reconnecting, show access message, and leave the chat route.
Unmount/logout	Unsubscribe and disconnect. Cancel reconnect timers and pending message requests.

7.2 Destinations and payloads

Use	Destination	Payload
Group send	/app/chat.sendMessage	{ groupId, content }
Group receive	Authorised group topic	Message id, groupId, senderId, content, sendAt
Listing send	/app/listing.sendMessage	{ conversationId, content }
Listing receive	Authorised listing conversation topic	Message id, conversationId, senderId, content, sentAt
Errors	/user/queue/errors	Human-readable safe error

Do not expose a UI that allows arbitrary topic input. Build topic names solely from the route's authorised group or conversation ID.

8. Recommended implementation checklist

- Central API client with request interceptor, one-time refresh interceptor, typed errors, request cancellation, and no token logging.
- Typed request/response models generated from, or checked against, deployed OpenAPI documentation.
- Reusable FormField, ErrorBanner, LoadingButton, EmptyState, PaginatedList, and ConnectionStatus components.
- Chat store keyed by conversation/group ID with message deduplication, stable ordering by timestamp/id, bounded in-memory page retention, and cleanup on unmount.
- Analytics and monitoring filters that redact Authorization, OTP, refresh token, email, and message body values.
- Tests for invalid OTP, token rotation, 401 retry once, 409 group/interest race, pagination, forbidden chat access, disconnect cleanup, and marketplace gateway routing.

9. Contract assumptions to confirm before frontend release

These are intentionally explicit because the backend implementation was not re-inspected for this document.

Topic	Confirm in deployed API documentation
Pagination	Query parameters, response envelope, sort order, maximum page size, and cursor stability for listings and histories.
Refresh	Whether refresh returns a rotated token pair, old-token invalidation behaviour, and 401/429 response bodies.
Marketplace mutations	Exact method/path for mark-as-sold and delete, plus inactive-listing response status.
WebSocket	Gateway versus direct service URL, broker destination names, heartbeat/reconnect recommendations, and access-error delivery.
Validation	Maximum lengths, accepted enum values, date timezone rules, and canonical field-level error format.

End of updated Frontend Build Guide - SquadStation